

If you want to match for only alphabets, then you can use the *alpha* construct. A few examples are shown below:

```
(rx alpha)
"[:alpha:]"

(s-matches-p (rx alpha)
  "ABC")

t

(s-matches-p (rx alpha)
  ";;")

nil
```

You can search for a blank character using the *blank* construct. For example:

```
(rx blank)
"[:blank:]"

(s-matches-p (rx blank)
  " ")

t

(s-matches-p (rx blank)
  "A")

nil
```

The *space*, *white* and *whitespace* rx-constituents can be used to match for white space as illustrated below:

```
(rx space)
"[:space:]"

(rx white)
"[:space:]"

(rx whitespace)
"[:space:]"

(s-matches-p (rx space)
  " ")

t

(s-matches-p (rx space)
  "abc")

nil
```

The *punct* construct is used to match for punctuation marks. A couple of examples follow:

```
(rx punct)
"[:punct:]"

(rx punctuation)
"[:punct:]"

(s-matches-p (rx punct)
  "abc")

nil

(s-matches-p (rx punct)
  ".")

t
```

You can match for a word using either the *word* or *wordchar* constructs. For example:

```
(rx word)
"[:word:]"

(rx wordchar)
"[:word:]"

(s-matches-p (rx word)
  "the")

t

(s-matches-p (rx word)
  " ")

nil
```

If you do not want to match for a word, you can use the *not-wordchar* construct as follows:

```
(rx not-wordchar)
"\\W"

(s-matches-p (rx not-wordchar)
  "abc")

nil

(s-matches-p (rx not-wordchar)
  " ")

t
```

The *repeat* construct takes two arguments: a number N and an s-expression. It repeatedly applies the s-expression N number of times to generate the regular expression. In the following example, we match for two occurrences of the letter 'x':

```
(repeat N SEXP) ;; Syntax

(rx (repeat 2 "x"))
"x\\{2\\}"

(s-matches-p (rx (repeat 2 "x"))
  " ")

nil

(s-matches-p (rx (repeat 2 "x"))
  "xyz")

t
```

You can use *group* or *group-n* rx-constituents to capture groups of regular expressions. The first argument to *group-n* represents the group number, which is followed by the actual s-expression. In the following example, we create a regular expression to match the date in the MM-DD-YYYY format:

```
(group SEXP1 SEXP2 ...) ;; Syntax
(group-n N SEXP1 SEXP2 ...) ;; Syntax

(setq mm-dd-yyyy
  (rx (group-n 3 (repeat 2 digit))
    "_"
    (group-n 2 (repeat 2 digit))
    "_"
    (group-n 1 (repeat 4 digit))))

(s-match-strings-all mm-dd-yyyy "12-10-2019")
(("12-10-2019" "2019" "10" "12"))
```

The generated regular expression pattern is `'\\(?:3(?:[:digit:]]\\{2\\})\\)-\\(?:2(?:[:digit:]]\\{2\\})\\)-\\(?:1(?:[:digit:]]\\{4\\})\\)'`.

You are encouraged to read the information given in <https://github.com/typester/emacs/blob/master/lisp/emacs-lisp/rx.el> to know more about the available constructs provided by rx.el. 

 By: Shakthi Kannan

The author is a free software enthusiast who blogs at shakthimaan.com.